

UNIVERSITI POLY-TECH MALAYSIA

PROJECT TITLE

MUHAMMAD ADAM MAHFIZ BIN ROSHAYULIZAM

**BACHELOR OF INFORMATION TECHNOLOGY(HONS) IN
CYBERSECURITY**

UNIVERSITI POLY-TECH MALAYSIA

Faculty of Computing & Multimedia

AI-Based Threat Level Classification for Network Logs

MUHAMMAD ADAM MAHFIZ BIN ROSHAYULIZAM
(AM2311015306)

FYP4074

MAY 2025

Declaration of Originality

This project is all my own work and has not been copied in part or in whole from any other source except where duly acknowledged. As such, all use of previously published work (from books, journals, magazines, internet, etc.) has been acknowledged within the main report to an item in the References or Bibliography lists.

I also agree that an electronic copy of this project may be stored and used for the purposes of plagiarism prevention and detection.

Copyright Acknowledgement

I acknowledge that the copyright of this project and report belongs to Universiti Poly-Tech Malaysia.

Signed :

Date :



Office Stamp

Abstract

This project addresses the critical challenge of network threat detection for resource-constrained organizations by developing an AI-Based Network Threat Classifier. To combat the inefficiency of manual log analysis, a Random Forest machine learning model was trained on the NSL-KDD dataset to automatically categorize threats into Low, Medium, and High severities. The primary achievement is a functional prototype featuring a Flask-based web dashboard that provides accessible data visualization and report generation. The system's reliability was successfully validated using standard metrics, including accuracy, precision, and F1-score. In conclusion, this work delivers a lightweight, effective, and academically-grounded cybersecurity tool that successfully democratizes access to AI-driven threat intelligence.

Table of Contents

1. Chapter 1: Introduction 9

1.1 Introduction..... 9

1.2 Project Background 10

1.3 Problem Statement 11

1.3.1 Overwhelming Volume of Unprioritized Network Logs 11

1.3.2 Manual Log Review Is Slow and Error-Prone..... 11

1.3.3 Lack of Lightweight AI-Powered Security Tools 11

1.4 Project Objectives 12

1.4.1 To develop an AI-based system to automate classifying network log threats 12

1.4.2 To evaluate the system’s classification performance using standard data science metric 12

1.4.3 To implement a web-based dashboard for visualizing threat classification and enabling the export of analysis report 13

1.5 Scope and Target User 13

1.5.1 Project Scope 13

1.5.2 Product Scope 14

1.5.3 Target User..... 15

1.6 Overview of This Report 16

2. Chapter 2: Literature Review 17

2.1 Scope and Target User 17

2.2 Investigation 18

2.3 Related Works..... 19

2.4 Comparison..... 20

2.5 Discussion..... 21

2.6 Conclusion 22

3. Chapter 3: Methodology 22

4. Chapter 4: Requirements 26

5. Chapter 5: Analysis 34

6. Chapter 6: References 59

Lists of Figure

Figure 1 Agile Figure (Medium, 2020)	23
Figure 2 Visual Studio Code (Canonical,2019)	30
Figure 3 Pandas (Sandeep, 2025).....	31
Figure 4 Flask (Sandeep,2025)	32
Figure 5 Neon Database ((Neon Serverless Postgres — Ship Faster, n.d.)).....	33
Figure 6 Role Chart.....	35
Figure 7 (Usefulness of the system)	36
Figure 8 (Necessity of the System)	36
Figure 9 (Frequency need to monitor).....	37
Figure 10 (Respondent' expectation part 1)	38
Figure 11 (Respondent expectation part 2)	38
Figure 12 (Importance of Explainability)	39
Figure 13 (Use Case model)	40
Figure 14 (User Flowchart)	42
Figure 15(User Flowchart after update Dashboard chart)	42
Figure 16 (Data Flow Diagram).....	43
Figure 17(Data Flow Diagram Left Side).....	43
Figure 18(BPMN Model)	45

List of Tables

Table 1 (Comparison Table)..... 21

Table 2 (Functional Requirements) 28

Table 3 (Non-Functional Requirement) 29

Table 4 (Hardware Table)..... 29

Acknowledgements

To begin with, I would like to thank the supreme Allah S.W.T who gave me the strength, patience, and perseverance, in the process of carrying this Final Year Project.

My deep honorable appreciation goes to my loving parents whose contribution in terms of support, encouragement and constant prayers have always been a source of strength in my academic life.

A special thanks to my Final Year Project Supervisor [Insert Supervisor Full Name] whose good advice, helpful comments and unparalleled assistance helped me in the creation of this project. I thank also the person in charge of the Final Year Project and other academic members of Universiti Poly-Tech Malaysia (UPTM) who also facilitated the process of the project.

I would like to extend my sincere appreciation and gratitude to my client Mrs. Abirami A/P Kunasagaran of Rohas-Euco Industries Bhd, to whom I interviewed due to her time, insight and interest in helping me complete this project although it was done in the constrained areas of integration as a result of company policy.

Friends and other students, I wish to thank you all in encouraging me, helping me to get moral support and giving us the experience to make this a less hectic trip and also a pleasure.

Finally, I would like to show thanks to Majlis Amanah Rakyat (MARA) that helped during my studies and offered me educational opportunities.

Without all these individuals and organizations in support, this project would have not been a reality. Thank you.

1. Chapter 1: Introduction

1.1 Introduction

The digital era is characterized by more complex and more massive cybersecurity threats. Malicious Actors may attack any organization of any size at any point, however, they may do it through a variety of ways .variety of methods, ranging widely between basic phishing messages and well-organized ransomware attacks.

Network monitoring systems are deployed by companies as normal security measures. intrusion detection devices to document and track network activity. There is however much difficulty in perusing this log information and determine what threats are actual and what are simply a false alarm, or normal traffic.

It is possible to compare Security Information and Event Management (SIEM) tools and traditional log analysis systems tend to be repetitive with alerts and many of them are not very serious. This occurs, when analysts receive an excessive amount of notifications, and fail to notice some crucial warnings. Also, manual analysis of logs is time consuming and error prone, particularly where log integrity is valued where not so many people have IT or cybersecurity expertise.

This proposal recommends the creation of a system that is based on AI to determine the level of threat on the network logs. This is a machine learning-based system that intelligently classifies the log data of the network traffic into three levels of severity: low, mid and high. The general idea is to create a light, simple to work with solution to small companies or schools that might not be able to afford money nor infrastructure of synthetic SIEM software. The aim of the system is to ensure that log analysis tasks are quickened. Less prone to errors, effective, and highly efficient through automation and intelligent detection of the threats.

1.2 Project Background

In the modern interdependent online world, Small to Medium-sized Enterprises (SMEs) and educational organizations have to deal with an unequal cybersecurity issue. The small size organizations usually face the same advanced threats as big companies yet they often lack both financial support and expertise in deploying and maintaining complex, enterprise-level Security Information and Event Management (SIEM) solutions. This resource shortfall leaves a very large gap in the entire cybersecurity environment.

The manual analysis of the network logs is one of the key operational burdens of the teams of IT workers in these organizations. Industrial network infrastructure and security instruments produce an uninterrupted and large amount of data, such as firewall alerts, system events, and traffic logs. Unless security personnel are smart enough to sort out the entries, thousands, or millions of entries need to be manually searched to point out possible dangers. Not only is this process ineffective and time-wasteful but also is highly prone to human error, resulting in a development termed as alert fatigue in which the actual threat can be missed among a multitude of harmless notifications (Kent & Grance, 2006).

Artificial intelligence (AI) and machine learning (ML) are the strong and scalable solution to this issue. With carefully-selected cybersecurity datasets, properly-trained models can get an AI system to self-learn network activity patterns to present itself armed with self-learned normal and malevolent trends to isolate. This project will fill the divide created by the existing large-scale solutions in the enterprise and the unaddressed needs of organization that lack resources. It suggests that this will be done by developing a lightweight, effective, open-source artificial intelligence that would be used to perform automated threat classification and thus successfully allow a smaller group of people to adopt a defensively superior stance in an efficient and cost-effective fashion.

1.3 Problem Statement

This project is based on the following main problems which have been discovered by prior research. As well as talking and talking to the client:

1.3.1 Overwhelming Volume of Unprioritized Network Logs

Modern network produces thousands or even millions of log entries each day. infrastructures. Firewall warnings, failed log-in, file access and system A couple of examples of these logs are errors. Unfortunately, most of these logs do not have a threat severity classification. Responses prioritisation is a difficult task among security teams because they are most of the times overwhelmed with the weight of having to inspect all the logs equally. because of alert fatigue that is caused by this torrent of information analysts are more prone to overlook some significant data accidents due to them becoming immune to warnings.

1.3.2 Manual Log Review Is Slow and Error-Prone

- Small organizations often carry out log analysis manually because there are no automated tools in businesses. Security analysts should go through logs, identify anomalies, and conclude threats that are associated with a manual review. Besides the time-consuming nature, the technique is easy to be erroneous by people. Research reveals that cybersecurity breaches can take place due to 95 percent of all breaches are human error. Cyberattacks of this day and age are simply too fast and complex so that it can be managed manually which brings more risk exposure.(Reason, 2016).

1.3.3 Lack of Lightweight AI-Powered Security Tools

Even though Microsoft Sentinel and other AI-driven SIEM solutions offer smart threat prediction, they will become more effective with assistance.

Small organisations often evaluate them as too expensive and complicated in terms of detection capabilities. Lightweight, simple, AI-based systems which are easy to install in low-resource or

The market is evidently lacking education settings. Very few tools are manufactured to hit an equilibrium between feasibility, simplicity, and performance. It is due to this reason that many small companies continue to use inefficient or non-existent processes.

1.4 Project Objectives

The realization of this project is predetermined by the next three basic goals, each of which is aimed at solving the identified problems and corresponding to the needs of the target user:

1.4.1 To develop an AI-based system to automate classifying network log threats

This is the main goal which deals directly with the issue of log volumes that are seemingly too much to handle and unprioritized. The task is to increase both sections to eliminate manually conducted, time-consuming review and develop a powerful classification engine. It hinges on going through the entire data science workflow: raw logs have to be cleaned so that they can be fed into an ML model, features need to be engineered to identify the most informative ones about the problem which we are solving, and finally, a Random Forest model ought to be trained that will be able to see the sophisticated patterns. The end product will be an intelligent system which will have a defined severity (Low, Medium, High) to the network event in real time which will enable the security teams to address most essential threat at once.

1.4.2 To evaluate the system's classification performance using standard data science metric

This goal is aimed at addressing the issue that manually reviewing process is time-consuming and inaccurate. In order to demonstrate the superiority of the AI-driven

method, the reliability of the system is to be measured quantitatively. It will be achieved by a thorough testing of trained model on a holdout part of the NSL-KDD dataset. A confusion matrix will be used to measure performance in terms of accuracy, precision, recall and the F1-score. Such measurements will yield an objective, scientific baseline of the performance of the classifier, and guarantee that it provides a more reliable and consistent analysis than human error-prone analysis.

1.4.3 To implement a web-based dashboard for visualizing threat classification and enabling the export of analysis report

The next goal relates to the deficiency of light and easy to reach AI security tools. The idea is to provide a highly useable front-end, as opposed to building some sort of elaborate system that would necessarily need some special knowledge. This will be through developing an application as a web based application making use of the Flask framework. The deliverable is not only an interface but also a functioning dashboard that provides the outcome of the analysis in form of intuitively understandable charts and tables. This makes the insights easy to interpret, at least to non-technical employees. Moreover, there is now an option of exporting a report that would offer a concrete product to record, use in a compliance or address in an even more extended dialogue.

1.5 Scope and Target User

1.5.1 Project Scope

The primary goals of this project shall be the design, the development, the testing of such an AI-based network threat classification system, and the evaluation of the same. The system will accept log files of network monitoring tools.

- Dissect the log files and get relevant information.
- Label all the log entries into one of three categories of threats.

- Present in a clear interface the results.
- Benefit usability, precision and performance.

Within the scope are performance testing in simulation conditions and environments, designing user interfaces, training of models. The project will not address such features as real-time monitoring, integration with enterprise SIEM tools, large-scale deployment. However, so as to accommodate the future upgrading, the system will be designed to have extensible and modularity.

1.5.2 Product Scope

The developed AI-Based Network Threat Classifier is a product of this project intended to identify and classify network traffic-related cybersecurity threats and include it into the process of supervised machine learning. The system will help users especially in small to medium scale businesses (SMEs) and academic institutions to figure out and evaluate network threats with minimal technical details involved.

This system will use NSL-KDD dataset to train machine learning framework and thereby being able to categorize different kinds of network attacks like Denial of Service (DoS), Probe, Remote to Local (R2L), and User to Root (U2R). The identified cases will be reported with a threat severity level (Low, Medium, High), which can be further used by the user to prioritize the matter and plan the response.

In order to make it accessible and usable, the system will have light interface such as command-line (CLI) or a very simple web-based UI. They will be in a position to load or enter log files and get the results of classification without knowledge of cybersecurity tools in advance.

Core Functionalities:

- update to accept input of log files in a structured format (e.g. CSV)
- Preprocess and extract features of input data

- Classify threat using a trained machine learning model The type of threat and the degree of the severity of every log entry output
- Provide display assessment measures of accuracy, precision, recall and F1-score (to validate performance).

Out of Scope:

- Real time traffic tracking or live packet capturing
- Production network environments deployment
- Automated response or fix-it actions

1.5.3 Target User

The target user is Mrs. Abirami Kunasagaran who is an expert in IT Department of Rohas Euco Industries Bhd. She is a user that monitors network security of a small and medium-sized businesses since she is in charge of managing cybersecurity activities.

- It needs tools that can assist with prioritizing incidents.
- lack of access to high priced commercial threat detection software.
- Require systems, which are easy to use and comprehend.

Secondary users may include:

- IT workers at training centers, colleges, or universities.
- The learners in cybersecurity are being taught on the use of studying logs data
- Security analysts at those companies that have little resources.

Although the system offers insightful information to rectify cybersecurity decision-making, it is developed in such a way that those with low levels of technical knowledge can use it.

1.6 Overview of This Report

All the stages of planning, development, implementation and measuring of the system AI-Based Threat Level Classification of Network Logs are discussed in details in the chapters which constitute this report. The outline below shows the content and objective of each chapter in brief:

Chapter 1: Introduction

The chapter is intended to give a summary of the project stating the history of the system development and reasons why it was developed. It attracts the attention to the statements of the problems, objectives, scope of the project and target public. It prepares the whole project in telling what should be done and who it should be done to.

Chapter 2: Literature Review

The chapter offers the detailed consideration of the related topics, including the numbering systems of threats, the use of AI in cybersecurity, and popular datasets, including the NSL-KDD. It compares the available solutions and reasons as to why the technologies and techniques that were selected in this project were suitable.

Chapter 3: Methodology

In this section, the development strategy, more precisely, the use of Agile methodology is outlined. The stages involved in the project i.e. the planning, design, development, testing, deployment and the review is articulated, besides how each of them assists the system to achieve its objective. goals.

Chapter 4: Requirements

This chapter has the list of functional and non-functional requirements of the system. It also describes the methods of gathering information about clients and users, so that the way the developed system meets actual needs is ensured.

Chapter 5: Analysis

Models/collected data including business process models, system flowchart and use case diagram are analyzed in this section. It provides a systematic description of how users will interact with the system and it has to be.

2. Chapter 2: Literature Review

2.1 Scope and Target User

This chapter makes an account of the existing literature and relevant concepts that are used to develop a background of an AI-based threat classification system against network logs. The overall goal of this chapter is to establish a theoretical and practical framework of the project due to the reasons, expressed in chapter 1. The review is going to begin with the more general issue of Artificial Intelligence (AI) in the modern realm of cybersecurity that involves a shift to smart and intelligence-based strategy abandoning the classical security concepts.

Here, the review will start to focus the attention to the Network Intrusion Detection Systems(NIDS), which is the immediate focus of the current project, and the information regarding the approaches of such systems as well as the review of the NSL-KDD training dataset that has been chosen as the training data to process by the proposed model. An analysis of the related existing works will discuss three of the available forms of solutions, namely: classical machine learning solutions, advanced deep learning solutions and commercially available Security Information and Event Management (SIEM) solutions.

Through proper comparison and discussion, this chapter will eventually point at the high unmet market requirement of a lightweight, usable, and robust AI security tool that can be used by SMEs and educators in general. The last section of this chapter shall draw conclusions to all

the findings to explain why and how this is an important initiative being undertaken to offer practical solution to address this gap.

2.2 Investigation

In order to come up with effective and practical AI-based threat classifier, three main parts have to be studied: the progression of intrusion detection, the importance of AI in cybersecurity and dataset.

Traditional Intrusion Detection Limitations

The more traditional Intrusion Detection Systems (IDS) and signature-based ones to be more specific, work by comparing network traffic patterns with a database of incident signatures that are known. These systems are efficient against the past known threats because they are reactive and thus cannot be used against zero-day attacks which are new exploits that cannot be described (Sculley et al., 2011). Since cyber threats are increasingly becoming high in volume, high in complexity, and high in variability, the impotency in these traditional tools is a drawback of serious concern.

The Role of AI and Supervised Learning

AI, specifically Machine Learning (ML) offers a scalable, intelligent alternative tool to signature-based tools. It is possible to train a ML model on historic data, to identify patterns, mark anomalies and forecast threat levels in real-time. Supervised learning falls in the category of ML approaches best-suited to the proposed project since it allows categorizing accurately on the basis of labeled training data (e.g. : DoS, etc.).

The project aims at generating detection as well as classifying the detections as levels of severity (e.g. Low, Medium, High) based detections that can make sense as prioritized threats, this makes sense to non-technical users. This systematic output is facilitated through the possibilities provided by supervised learning, which maps one-known-input (log features), to a desired output (threat category).

Dataset Selection: NSL-KDD

The decision of a dataset is vital. NSL-KDD dataset is chosen based on its relevance, format and broadly accepted in the literature of cyber security (Tavallaei et al., 2009). It offers:

- 41 characteristics on every record of network connection (e.g. duration, type of protocol, service, flag status)
- Namings of classes (ex.: "normal", "DoS", "R2L", "U2R", etc.)
- Pre-balanced records, solving duplication problem of the old KDD99 dataset

The data format allows scaling up model training in addition to features engineering.

2.3 Related Works

In order to properly situate this project in the wide spectrum of the study of network security, three archetypes of Intrusion Detection Systems (IDS) were reviewed comparatively. The described analysis explains the targeted technological and economic gap to be closed by such a project.

➤ **Traditional Signature-Based IDS (Snort)**

Snort is an early open-source IDS whose major action mechanism is signature-based. Its approach is the scan of coursing information packets in the network and comparing the information contained in these packets with a large database of previously defined rules, or signatures, that represent the known malware and attack patterns (Roesch, 1999). Although very speedy and effective in detecting well documented threats, the major drawback with Snort is its reactive character. It is in essence unable to warn of new and zero-day attacks or polymorphic malware where no signature has been developed yet and thus it is both a necessity and not enough as a security measure. (*SNORT - Network Intrusion Detection & Prevention System*, n.d.)

➤ **Modern Hybrid Open-Source IDS (Example: Suricata)**

The Suricata is the development of an open-source IDS. It has a big ruleset that matches signatures just like Snort, yet it adds some enhanced features. It is also multi-threaded and thus can exploit more modern multi-core processors to increase performance. Better still, it goes deeper with protocol analysis and can trace the entire flows within a network, bypassing what single-packet analysis would examine (Suricata-IDS, n.d.). Irrespective of these innovations, Suricata has not yet advanced to the point of having rule-based detection logic that is also native to machine learning and has the ability to detect previous unknown, anomalous behavior based on learned patterns of normal behavior excluded. (*Home - Suricata*, 2025)

➤ **Commercial SIEM Platforms (Splunk)**

High-end solutions are commercial Security Information and event Management (SIEM) platforms such as Splunk. Such systems act as end-to-end data analytics solutions and take in massive data across entire organizations. They are capable of conducting advanced behavior analysis, match related but unrelated events to detect complex attack campaigns and have powerful forensic tools using the sophisticated AI and machine learning applications (Splunk, n.d.). Nevertheless, they bring a lot of operating expenses with the power. Not only is Splunk expensive to get and maintain with a significant licensing cost, high-performance server infrastructure, and a battalion of highly-trained security analysts to operate, but it is also far out of reach in terms of both cost and human resources budget of SMEs and academic establishments. (Splunk Inc., n.d.).

2.4 Comparison

Comparison of similar products shows that there is a clearly defined and unmet gap in the market in terms of cybersecurity. The architecture will work to fill in the void between the rudimentary signature-only tools and the enterprise level tools, which are unreasonably expensive. Comparison table is as below.

Feature	Proposed System (This Project)	Snort	Splunk (Enterprise Security)
Core Technology	Supervised Machine Learning (Random Forest)	Rule-Based Signature Matching	Full-Spectrum AI/ML, Anomaly Detection, Event Correlation
Detection Method	Learns patterns from historical data to classify threats	Compares traffic to a database of known attacks	Baselines normal activity and detects complex deviations
Cost Model	Free, Open-Source	Free, Open-Source	High Subscription & Infrastructure Costs
Target User	SMEs, Students, General IT Staff	Dedicated Network/Security Engineers	Security Operations Center (SOC) Analysts
Key Advantage	Accessible, low-cost AI-driven threat prioritization	High-speed, low-resource detection of known threats	Extremely powerful and comprehensive threat hunting
Key Limitation	Batch processing of logs (not real-time)	Ineffective against novel (zero-day) attacks	Prohibitively expensive and complex for smaller organizations

Table 1 (Comparison Table)

2.5 Discussion

Having looked into the available literature and compared them, a critical need gap emerges in the cybersecurity field: whereas lots of solutions with high performance can be found, very few approaches are simplistic, resource-efficient, and user-friendly.

In the majority of scholarly efforts, accuracy metrics are concerned, and the resulting systems remain not deployable. Likewise, the high end tools being used are very strong but cannot be financially or technically supported by the smaller organizations or universities.

The proposed system fills this gap by :

- Since an adequate level of accuracy and interpretability is required, a proven ML model (Random Forest) is used
- Runs on low resource requirement and fits laptops/VMs
- Offering a convenient interface (CLI/Web) of input of logs and feedback on the threat
- Customisation of the output according to level of severity (Low, Medium, High) to action prioritise

This seems to it that is not only a technically working system but also a practically busy system.

2.6 Conclusion

This relative study comes up with a conclusion that the proposed project is at a unique position. It offers the smarts of machine learning, something that is lacking in conventional open-source solutions, and lacks the excessive complexity and expense of commercial SIEM software. It establishes a niche as an effective and affordable entry point into an AI-enabled security system to companies that are already underrepresented on the market.

3. Chapter 3: Methodology

3.1 Introduction

This chapter offers the exhaustive description of methodology used to run the project of AI-Based Threat Level Classification of Network Logs. It describes the particular processes, project management structure and methods to be applied to achieve the targets of the project in an orderly and effective way. The aim of integrating a formal approach is to ascertain flexibility to the intricacies involved in machine learning development and have the end output to be in accordance to the client specifications. Every detail of the methodology will be analyzed in connection with the particular objectives of this project which are to develop a threat classifier, create an interface, which is easily accepted by the user, and improve the system behavior.

3.2 Agile Methodology

Agile methodology will take charge of the development of the AI-Based Threat Classifier. The current project management concept focuses on the iterative development that breaks down the projects into small maneuverable stages termed as sprints. (Agile Alliance, 2021)



Figure 1 Agile Figure (Medium, 2020)

There are a number of convincing reasons why the Agile methodology would be a perfect fit in this project:

- **Iterative and Flexible Process:** The very nature of a machine learning development is experimental. A roll out method can be used to carry out successive improvements to the classification model and front-end interface through the results of testing and viewpoints of stakeholders.
- **Stakeholder Collaboration:** Agile also encourages direct contact with the project client, Mrs. Abirami Kunasagaran of the Rohas-Euco Industries Bhd. This makes sure that the system is in a continuous cycle of development so that it can sate the practical needs of her department.
- **Concentrated Growth:** It is possible to solve rather complicated tasks in limited sizes working in brief, concentrated sprints because in this way, one can be sure that the project is divided into small steps and is steadily developing, enabling the project to respond swiftly to any changes or shifts in needs that may occur.

3.3 Phases in Agile Methodology

This project will adopt the Agile development process that is divided into a few major phases. It is a logistic approach to work as each phase is a homogeneous stage of work and there is even a beginning and an end of each work.

3.3.1 Plan

The project is based on the planning stage. In this phase the scope, objectives and requirements of the project forming it are defined formally. This implies the in-depth research of the problem area and identification of concrete requirements on the part of the client.

- Activities: Interview clients, search the relevant data (e.g., NSL-KDD) and machine learning algorithms, and find accurate problem statements.
- Key Deliverables :
 - Project Scope Document
 - Proper Problem Statements
 - Background Research and Dataset choice
 - First contact notes with clients

3.3.2 Design

The design stage involves development of the architectural concept of the system. These are the technical architecture, the data flow diagrams and the user interface design. These will be modular and easily changeable because of client feedback hence the design.

- Activities: develop system architecture illustrations, wireframe targeted UI of the web-based interface, and trace the path of data flow between the stream of the logs to the representation values of the threat levels.
- **Key Deliverables:**
 - System Architecture Map
 - UI Wireframes/Mock-ups
 - Data Flowchart
 - Design Specification Document

3.3.3 Development

This is the implementation phase proper during which the prototype of the system will be created. It includes creating code of a machine learning model, backend logic of data processing and frontend user interface.

- Activities: Pre-process the NSL-KDD data set, perform training of the machine learning model with Python and Scikit-learn and prepare the web interface with Flask.
- Key Deliverables:
 - NSL-KDD Dataset ready to be preprocessed
 - trained Machine Learning Model (Prototype)
 - Working Threat Classification Engine
 - Web-based User Interface (Mock up)

3.3.4 Testing

Testing phase plays an important role in realizing the quality, accuracy and reliability of the system. A set of rigorous testing at different levels will be carried out to check the performance of the machine learning model and the functionality of the system as a whole.

- Activities : Apply unit tests on each independent functionality, integration tests to confirm that all segments interact and system tests using simulated log files. Accuracy, precision, recall, and F1-score will be considered when analysing the performance of the model.
- Key Deliverables:
 - Testing Unit, Integration and System Reports
 - Performance Report
 - Bug/Issue Tracker -Documentation

3.3.5 Conclusion

Finally, use of the Agile methodology that is planned in phases of planning, design, development and testing can offer a solid foundation to this project. This method is rather specific to handle the cyclical character of development of AI systems that should be flexible, continuous and it should be very close to the requirements of the client. These details about discrete stages and deliverables make the process of developing the AI-Based Threat Level Classifier not only disciplined, but also flexible. Although Agile is not necessarily a one-size-fits-all tool to use in every project, its principles have successfully been implemented in that matter at hand to ensure that a working and potent cybersecurity tool could be created successfully.

4. Chapter 4: Requirements

4.1 Introduction

The process of requirements gathering and definition is a key constituent of software development life cycle (SDLC). The requirements outlined in the current chapter are specific to the system of analysing the network logs with the use of AI-Based Threat Level Classification. The objective is to establish a transparent, definite, and checkable basis of the follow-up design, development, as well as the testing steps. This is done by the documenting of features and functionality of the application so that the resultant product provides requirement of the client in the application and/or the industry.

The needs fall into two major classes: functional and non-functional ones. Non-functional requirements set the way the system will execute its functionalities because they outline how the system will carry out its functions, mainly on factors that show performance, usability, and reliability. Both of the categories are described in this chapter giving a full idea of how the system is supposed to act and its features.

4.2 Data Gathering Techniques

A qualitative research and a technical investigation was used to ensure the requirements are comprehensive and precise representation of the needs of the project.

1. **Client Interview:** The main tool used in collecting user requirements was a simple face to face interview with the key stakeholder who happened to be the wife of the CEO of Rohas-Euco Industries Bhd who is Mrs. Abirami A/P Kunasagaran. This qualitative method was the necessary way to learn the real issues of working with the IT department, their current workflow, certain discomforts connected to the process of log analysis, and perspectives about the implementations of the automated variant of the work.
2. **Literature and Technology Review:** A thorough review of existing technologies, academic papers, and technical documentation was conducted. This served a purpose similar to competitor analysis by establishing the technical feasibility and identifying the best tools for the project. The key areas of investigation included:
 - **Machine Learning Framework:** Weka and Sklearn frameworks used to compare them and reveal the most appropriate library in order to apply the Random Forest model.

- Web Development Frameworks: Evaluating Flask and Streamlit in terms of its suitability to develop simplified and responsive user interface.
- Cybersecurity Datasets: A case study of the detailed analysis of the NSL-KDD dataset on its characteristics, format, and the effectiveness of the dataset to create a powerful model of intrusion detection.

4.3 Functional Requirements

Functional requirements are what components of the system can do and they are defined as behaviours. They entail basics that enable the user to gain an end result.

ID	Requirement Name	Description	Justification / Source from Flowchart
FR-01	User Authentication	The system shall provide a secure mechanism for users to register for an account and log in.	This is an implicit prerequisite for features like storing historical data and displaying user-specific trends.
FR-02	Log Data Submission	The system shall provide an interface for an authenticated user to submit log data, either by uploading a log file or by pasting raw text.	Directly sourced from the "User uploads log file or enters log text" step at the start of the flowchart.
FR-03	AI-Based Threat Classification	Upon data submission, the system shall automatically perform data processing, feature extraction, and apply the trained ML model to predict a threat level.	Sourced from the "Data Processing & Feature Extraction" and "ML Model predicts threat level" steps. This is the core analysis engine.
FR-04	Real-Time High-Severity Alerting	If a high-severity threat is detected, the system shall immediately emit a real-time alert to the user's dashboard (e.g., via Socket.IO).	Directly sourced from the "High Threat?" decision branch, which leads to "Emit Real-Time Alert" and the "Live Alerts Panel on Dashboard."

ID	Requirement Name	Description	Justification / Source from Flowchart
FR-05	Historical Threat Data Persistence	The system shall store a summary of all analysis results in a database to track threats over time.	Directly sourced from the "Store results in Database" step, which is crucial for trend analysis.
FR-06	Threat Trend Visualization	The system shall provide an API endpoint (/api/trends) and use it to display a historical trend line chart of detected threats on the user's dashboard.	Sourced from the entire "Threat Trends" pipeline in the flowchart, from the database update to the final line chart display.
FR-07	Exportable PDF Reporting	The system shall provide a function for the user to generate and download a formal PDF report summarizing the analysis findings.	Directly sourced from the "Generate PDF Report" and "User downloads/views report" steps.

Table 2 (Functional Requirements)

4.4 Non-Functional Requirements

Quality attributes of the system and characteristics of its operation, otherwise known as non-functional requirements, are developed. They explain the way functional requirements must be fulfilled.

Category	Non-Functional Requirement
Performance	The system should be able to process moderately sized log files and return classification results in a timely manner to be more efficient than manual analysis.
Usability	The user interface must be intuitive and easy to use, requiring minimal training, especially for non-technical IT staff.
Reliability	The system must provide consistent and accurate threat classifications. It should handle valid inputs without crashing or producing erroneous outputs.

Category	Non-Functional Requirement
Accuracy	The machine learning model must achieve a high level of accuracy, precision, and recall when tested against benchmark datasets to be considered reliable for threat detection.
Security	As a cybersecurity tool, the system itself must be secure, ensuring that uploaded log data is handled confidentially and is not exposed to new vulnerabilities.
Scalability	The system should be designed to handle an increase in log file size or user load without a significant degradation in performance.

Table 3 (Non-Functional Requirement)

4.5 System Requirements

This section contains the hardware and software requirements of both the system development and the end-user to make the development and use of the system clear in terms of what type of resources are needed in the full development cycle of the system.

In this subsection, describes the special tools and libraries as well as hardware resources that the developer needs in order to build, train, and test the system.

- **Operating System:** Windows 10/11, macOS, or a Linux distribution (e.g., Ubuntu).
- **Programming Language:** Python (Version 3.8 or higher).

Component	Specification
Computer	Personal Computer or Laptop
Processor (CPU)	Modern Multi-core CPU (e.g., Intel i5/i7, AMD Ryzen 5/7 or newer)
Memory (RAM)	Minimum 8 GB (16 GB or more recommended for handling large datasets)
Storage	At least 20 GB of free space for datasets, libraries, and the project environment.
Network	Internet access for downloading libraries, datasets, and conducting research.

Table 4 (Hardware Table)

4.5.1 Visual Studio Code



Figure 2 Visual Studio Code (Canonical,2019)

Visual Studio Code (VS Code) is a free, lightweight source-code editor created by Microsoft. It operates with various programming languages including Python, JavaScript, HTML/CSS, C++, and Java, indicating that a considerable number of developers utilize it (Microsoft, 2024).

Its great strength lies in its high customizability through extensions. The integrated marketplace (VS Code Marketplace, 2024) enables users to incorporate features such as AI code tools, themes, and GitHub integration, alongside supporting multiple languages. It includes a terminal, robust debugging tools, and facilitates developers to code, test, and collaborate in a single task.

VS Code is available at no cost and can run on Windows, macOS, and Linux systems, making it universal. This has turned it into one of the most widely used editors globally, known for its speed, adaptability, and range of features.

4.5.2 Pandas



Figure 3 Pandas (Sandeep, 2025)

Pandas is a basic Python library constructed to ensure that work on structured data turns out to be easy and extremely efficient. Fundamentally, the library also presents two unavoidable data structures, namely DataFrame, because it stores data in the form of a table of columns and rows with column names and Series, which describes one column of data. Users can easily scrub, model and massage data using these structures. Any routine action unfamiliar to the spreadsheet, e.g. row filtering by a given criterion, changing the cell values or solving some compound operations within the data is simplified as an understandable order.

Their role in the data analysis process may be understated, but it has to be considered that it is Pandas that is usually better at dealing with the formats of the data in the field since tabular data are overwhelming. It is professionally designed to ingest and deal with data sources such as CSV files SQL databases and Microsoft Excel spreadsheets. As a result, Pandas has since become the established industry library of choice when it comes to the much-needed initial steps of any project of a data driven nature such as data cleaning, data transformations and data search. It is the foundation stone that all down-stream analysis builds upon.

To describe how it would be beneficial to use in a real business application I will use an example of a spreadsheet containing crude sales data. Pandas would load this data into a DataFrame which would be analysed by an analyst whose duty is to draw conclusions. Based on that, they could easily do multiple operations without creating a complicated program, including calculating the average monthly earnings, narrowing down the overall range of data to see the results of a particular area, or establishing a trend in sales over time. Pandas enables the user to respond to such vital business questions effectively, making a stagnant data file an analysis tool. (The pandas development team, 2024)

4.5.3 Flask (Web-App-Development)



Figure 4 Flask (Sandeep,2025)

Flask is a python microweb framework and commonly termed as a micro-framework, it is small and simple. It has a major role of offering the tools required in developing applications in the web and Application Programming Interfaces (APIs). Instead of providing a large number of built-in features, Flask gives a stable framework of web request tracking, session management, and template execution. Its modular and simple design permits programmers to build an operative web service using very little preparatory setting up and coding.

The potential of Flask is that it provides great freedom and flexibility to its developers hence a great candidate to learn web development. Flask is not as prescriptive as more opinionated frameworks, instead it does not tie you to any particular organization or pattern of how to do something, such as interacting with a database or validating a form. The lack of opinions reduces the barrier to entry since developers will not be forced to learn an already complicated architecture before being able to build applications. This versatility also allows the developers to select the precise parts that suit their project adequately.

The most real-life instance of the usefulness of Flask is the development of a basic web application, including one that was used as an individual to-do list. With Flask, one would be able to configure URL routes to respond to user requests in no time, define Python functions to perform HTTP requests of adding or deleting tasks, and a templating engine will be used to dynamically display the to-do list in a web browser. There is very little and highly legible code required to achieve the entire core functionality of such an application, indicating the strength of Flask in fast prototyping and small projects development.(Grinberg, 2018)

4.5.4 Neon Databases



Figure 5 Neon Database ((Neon Serverless Postgres — Ship Faster, n.d.))

Azure Native Integrations qualify as a long-term bet by the Microsoft corporation to automatically deploy, administer and finely integrate customized software and services by other firms straight into the Azure platform. This strategy enables the customers of Azure to use the best-of-breed technologies, which they see the same way as the native Azure services. The Neon Serverless Postgres service is a highly-visible one, a collaborative service offered by both Microsoft and Neon, and hosted as well as maintained by both companies. It is easy to use the service by the customers in the way they open the Azure Portal and purchase the service in the Azure Market.

Neon is built into a cloud-native Postgres that is designed to meet the needs of the modern app. It provides serverless, server management and highly scalable databases. The system is characterized by an array of advanced functions, such as scaling the provided resources automatically, a scale-to-zero pattern to reduce spending when the resources are not in use, database branching allowing development and fault testing in isolation, a clear point in time restoration within seconds and time travel queries of past analysis. Neon isolates the physical database infrastructure, including configuration, maintenance and scale up of it, so that development teams can spend their energy and time building and enhancing their applications, not managing their databases.

The key advantage of this native integration is exceptionally thorough unification and streamlined ease of use of the Azure workloads. The service includes a smooth onboarding experience, in which developers are able to utilize capabilities of Neon in a short period. Managing-wise, everything Neon is provisioned and managed through the Azure portal itself, so it is listed on the same screen as any other native Azure resource of an overall resource list (All resources). In addition, the integration reduces administrative overheads by consolidating monthly billing using the Azure marketplace and single sign-on (SSO) functionality and removes the necessity of individual account management. This unifying strategy helps databases to be optimised in terms of performance and reliability in an infrastructure that is powerful, cost-effective, as well as fully integrated in Azure.

4.6 Conclusion

This chapter has entailed a thorough specification of the requirements of the AI-Based Threat Level Classification system. With non-functional and functional requirements being explicitly determined together with the system requirement, there is indeed an adequate and auditable basis of the project. The requirements will serve as guideline to the development team to undergo the following stages of analysis, design, and implementation to ensure that the ultimate product is effective, reliable, and that it is directly linked to that of the client.

5. Chapter 5: Analysis

5.1 Introduction

In this chapter, a very close examination of the information downloaded to develop the design of the system of AI-Based Threat Level Classification of Network Logs is done. It can be the decisive step between the abstract needs which are set in Chapter 4 and the solid system design. The first approach that will be followed in this case is a quantitative research of respondents that will provide the survey answers of a selected group of industry experts.

The information obtained on the basis of this analysis will help prove the hypothesis of the project, its clarity, and focus its goals and features. These proven user requirements will then be converted into official design deliverables: a Use Case Model to specify user interaction, a System Flow chart to show how the process works, and a Business Process Modelling Notation (BPMN) diagram to show the end- to-end business process flow. Such a hard science-like, evidence-based procedure promises not only to provide a smoothly technically sound result but also represents an immediate fit to the practical demands and expectations of the planned users.

5.2 Data Gathering Analysis

It was developed through Google Forms, and the questionnaire was shared among 57 respondents in different organization positions. These responses can be examined to give

the view of what the issues are like within the problem domain and to give a powerful mandate to the project.

1. Respondent Profile and Familiarity

The audience in the survey was also diverse, accounting to a significant percentage of respondents as being the "General Staff" (50.9%). A not insignificant and technically relevant group of participants was however captured by the profession of "Network/Security Engineers" (12.3%) and "IT Managers" (8.8%). Here it means that although the requirement of the security tools is across an organization, the responses are biased with the advice of the experts.

More importantly, 96.4 percent of the respondents had indicated that their knowledge of cybersecurity equipment was 3 or more on the 1-5 scale with 42.1 percent indicating that it was 4. This is to verify that audience is informed and their opinion is informed.

What is your current role in the organization?
57 responses

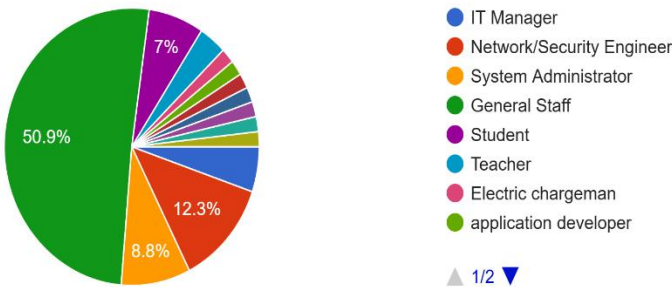


Figure 6 Role Chart

2. The Technology Gap

Based on the results of the survey, it is possible to note a huge disruption in terms of the current capabilities. Out of the percentage of 100 when queried whether their system offers automated classification of the threat, 33.3 percent answered that it is not sure with 24.6 percent also indicating that it has a system of no automated threat classification. It means that most users do not have the option (57.9%) or are not even aware about it, so it is an opportunity to establish for a tool that offers explicit and clear classification of the threat.

Does your current system provide automated threat classification?
57 responses

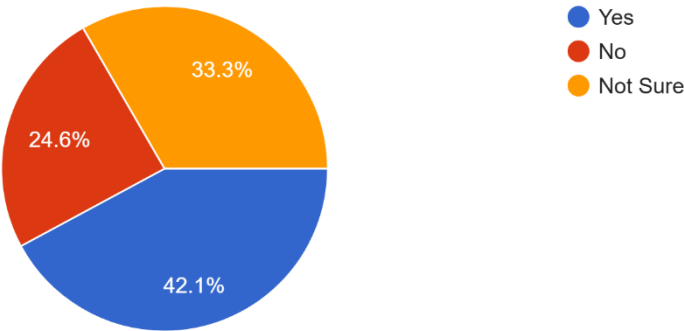


Figure 7 (Usefulness of the system)

3. Confirmation of Project Need and value:

The fundamental hypothesis of the project is well confirmed by the data. The vast majority (91.2%) of those who were asked think the concept of an AI-driven threat classification system will be useful, and 56.1% of them agree that it will be very useful, as 56.1 percent replied, Yes, definitely. This gives a strong and clear directive of the project. The necessity is also supported by the frequency of reviews of the logs, with the sum of 66.6 percent of respondents reading logs daily or weekly, which proves it a common and time-consuming activity.

Would an AI-based threat classification system be useful in your environment?
57 responses

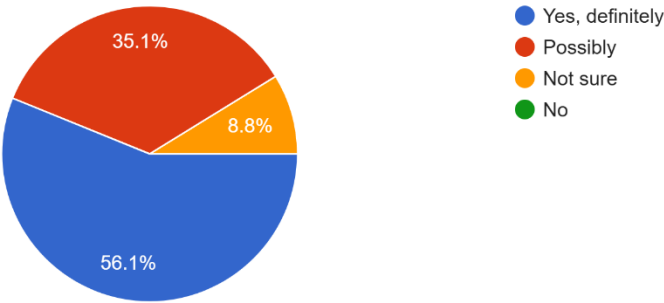


Figure 8 (Necessity of the System)

How often do you or your team review network logs or traffic data?
57 responses

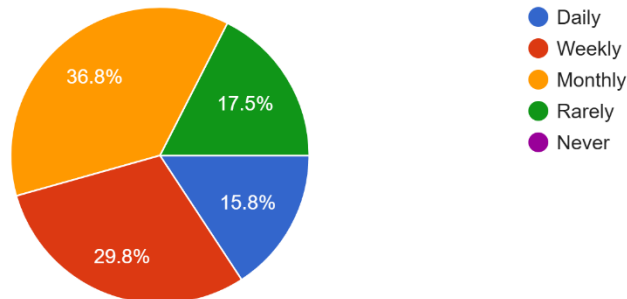


Figure 9 (Frequency need to monitor)

4. Interface and Feature Preferences:

A definite design course can be given by user preferences. The method of interaction with the strongest preference of 43.9% has been identified as Web-Based Interface, and therefore this has been identified as the main target of development. The high results of Command-Line Interface (26.3%) and "Either is fine" (26.3%) indicate that target users are professionally capable and would enjoy the extremely functional design rather than the mere beautiful design.

The most feature in demand was "Real-time threat alerts" (56.1%). It is a very important conclusion. Though analysis in real time is beyond the boundaries of the first version of this project (where attention is paid to log files uploading), such data is highly valuable. It makes clear that the concept of event-driven alerts is very popular and must be regarded as a priority future development or one of the priorities of the next-phase roadmap.

In the given project, it confirms that the main in-scope feature, namely, "Upload log files to analyze it," proved to be the second most frequently chosen unique feature. The willingness to provide such features as "Visual dashboards", "Historical report generation" also favor the decision of using web-based UI, as these are presented in the most convenient way using this type of UI.

Which type of interface would you prefer for interacting with the system?
57 responses

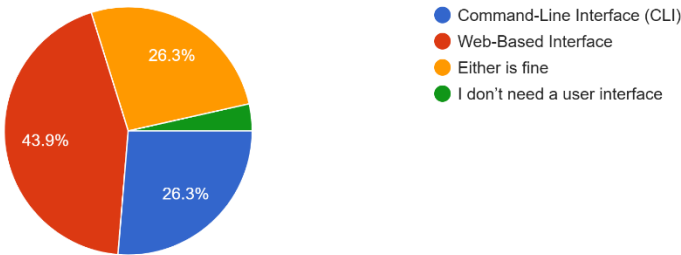


Figure 10 (Respondent' expectation part 1)

What features would you expect from a network threat classifier?
57 responses

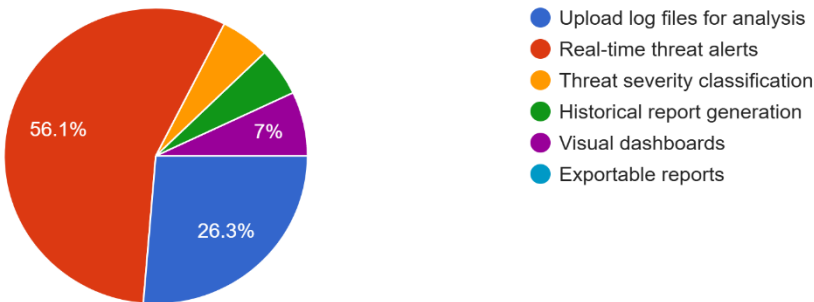


Figure 11 (Respondent expectation part 2)

5. Importance of Explainability

The number of users who found out that explainability (knowing why a threat is classified as high/low) is very important (a score of 4 or 5) scales up to 68.4%. This is one of the main non- functional requirements implying that the system cannot be the black box. Where possible, the results or the UI ought to have a context to some of its classifications.

How important is explainability in AI decisions (i.e., knowing why a threat is classified as high/low)?

57 responses

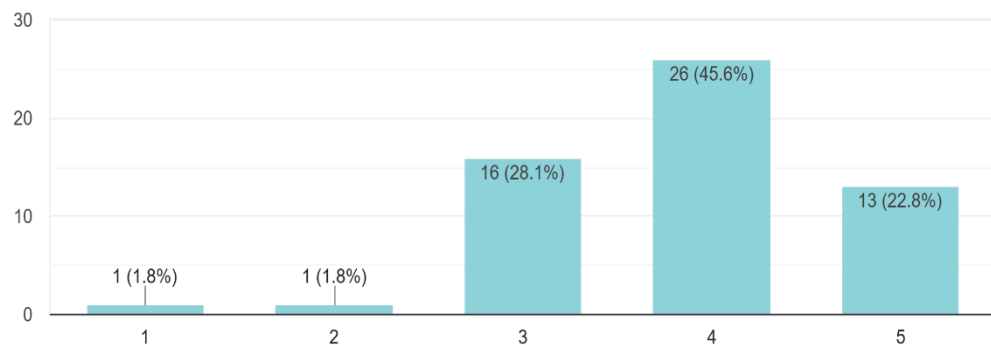


Figure 12 (Importance of Explainability)

5.3 Use Case Model

A Use Case model defines the behaviour of a system in the view of an user, and explains how an actor documents his communication with the system in order to deliver a particular outcome. This is the model that will be necessary in the establishment of the scope and main functional requirements. The IT/Security Professional is the major player of this system.

Use Case Components:

- Actor: IT/Security Professional (e.g. Network Engineer, IT Manager, Overall Staff with security duties).
- System Boundary: The Threat Classification System, which uses Artificial Intelligence.
- Use Cases:
 - Analyze Log File: This is the first use case. The actor gives the system a log file to work on it. This is directly related to the user expectation that the user can upload log files and analyzing it.
 - View Threat Dashboard: The actor uses an interactive dashboard to get a top-level summary of the analysis findings (e.g. number of threats, by severity). The preference of the user to utilize a "Web-Based Interface" and their expectation of obtaining "Visual dashboards" are immediately the factors that created this use case.

- Export Analysis Report: The actor may download or save the analysis output so he or she can review, document, or share offline. This supports the user desire of having the ability to Exportable reports.

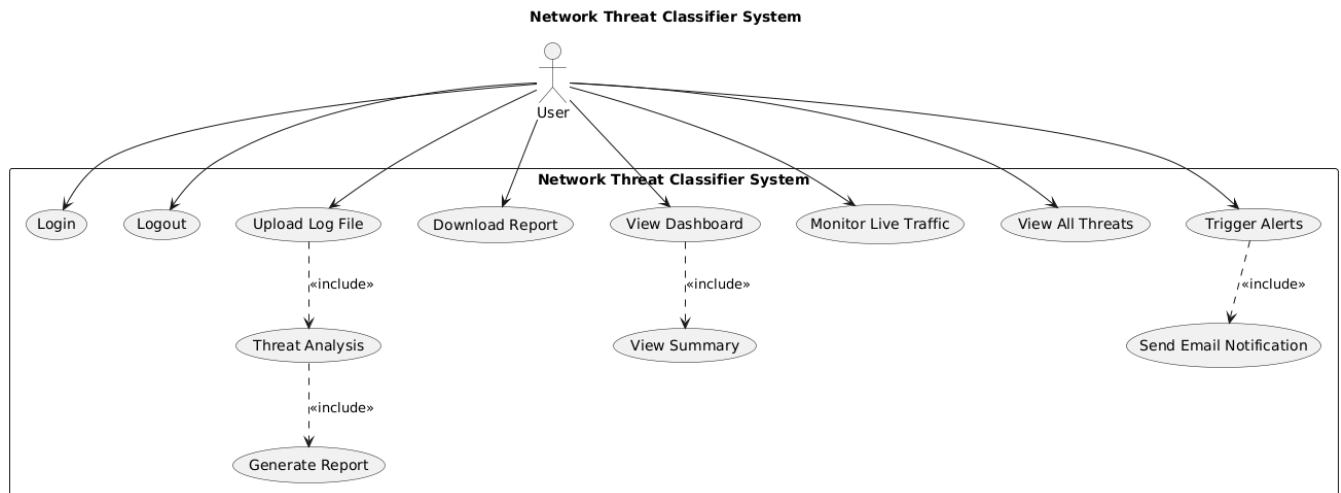


Figure 13 (Use Case model)

Diagram description: Figure 5.8 reveals how the IT/Security Professional relates with the core functions of the system. The actor is able to start the Analyze Log File procedure. The outcomes of this analysis are then used via View Threat Dashboard use case. Lastly, the user is able to Export Analysis Report, thus giving a visible result to record. The three use cases described convey the key and in-scope responses, which the survey confirms.

5.4 Flowchart

The flowchart illustrates the **data processing and alert generation workflow** for the **AI-Based Threat Level Classification for Network Logs** system. It shows how user input moves through the system, how the machine learning model predicts threat levels, and how the results are managed and displayed.

Flowchart Steps:

1. **Start**
The user gets into his session with the system.
2. **Upload log File**
By the user uploading a log file, the threat analysis can be done. It might be a regular log file or some structured data like NSL-KDD.
3. **Watch Traffic in Realtime**
Alternatively, the user can also opt to trace live network traffic thus initiating real-time processing.
4. **View Dashboard**
Once input has been processed, the user logs in to the dashboard to have access to summarized results of analysis such as the threat levels, and trends.
5. **Review Results**
The user looks at the results they are presented with like classification results, confidence scores as well as alerts sum up.
6. **End**
The system response is concluded when analysis and review is done.

This user flow guarantees that users will have a possibility to select between analysis of log files and live monitoring and easily access threat intelligence with a visual representation.

C

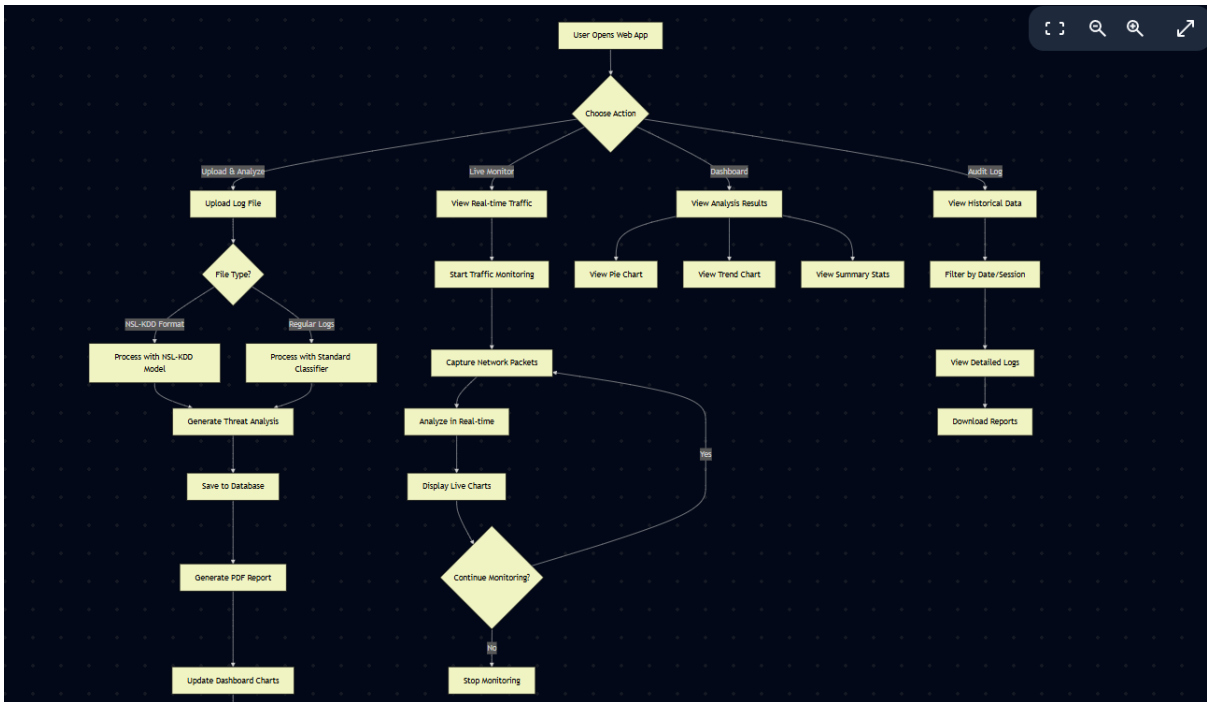


Figure 14 (User Flowchart)



Figure 15(User Flowchart after update Dashboard chart)

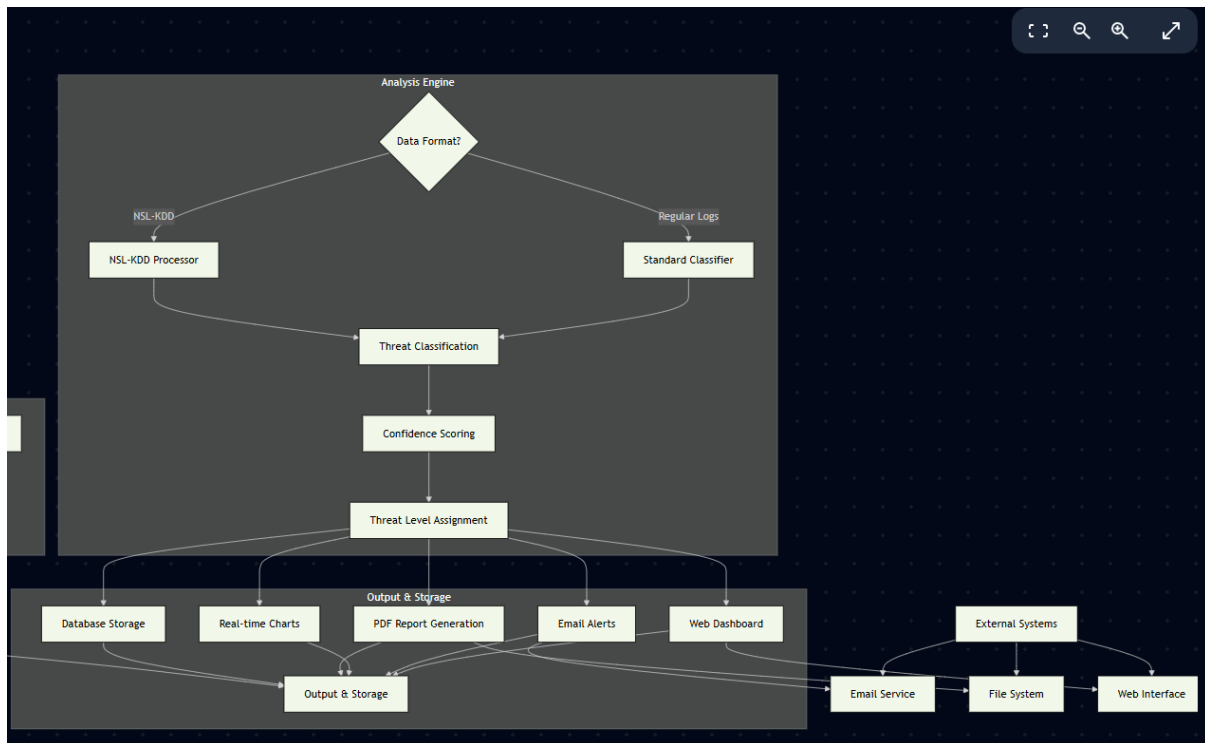


Figure 16 (Data Flow Diagram)

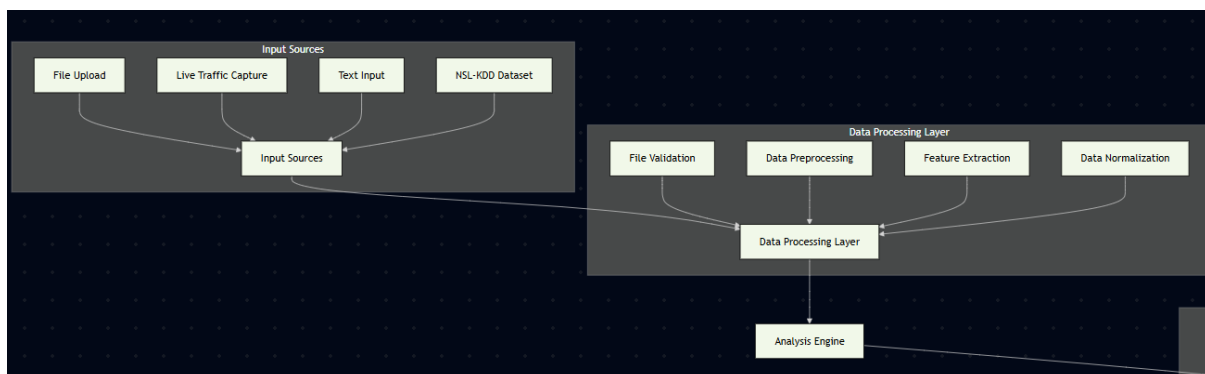


Figure 17(Data Flow Diagram Left Side)

The logic of the system is built in the layered model of the data flow in which every module has definite tasks. It is done as follows:

1. Input Phase

- The User downloads a log file or launches a live session.
- The input data is fed into the Web Server.

2. Validation Forwarding

- The Web Server runs a validation check on the file being uploaded in terms of format and integrity.
- The data will be sent into Machine Learning Engine after validation.

3. Stage of Processing (ML Engine)

- ML Engine does data pre-processing, feature extraction and classification model execution.
- Depending on the processed log, the model identifies the level of the threat (e.g., Low, Medium, High).

4. Result Handling

- These results are sent back to Web Server.
- The data is retained by the server, and PDF reports are produced, as well as visualizations on dashboards are updated.

5. Output Phase

- The User inspects the dashboard and gets alerts or reports.
- Additional actions that the user may perform include re-examining threats or initiate further investigation.

Just by glancing at this logical chart, one can identify a sorted, tiered architecture: the User Layer, the Web Application Layer, and the Machine Learning Layer all connect with each other using established data flows.

5.5 BPMN (Business Process Modelling Notation)

BPMN gives a graphic description of the business process in a chart. It is more formal than a flowchart and is superior at displaying the distinct division of duty among the various participants or systems, divided into lanes within a pool.

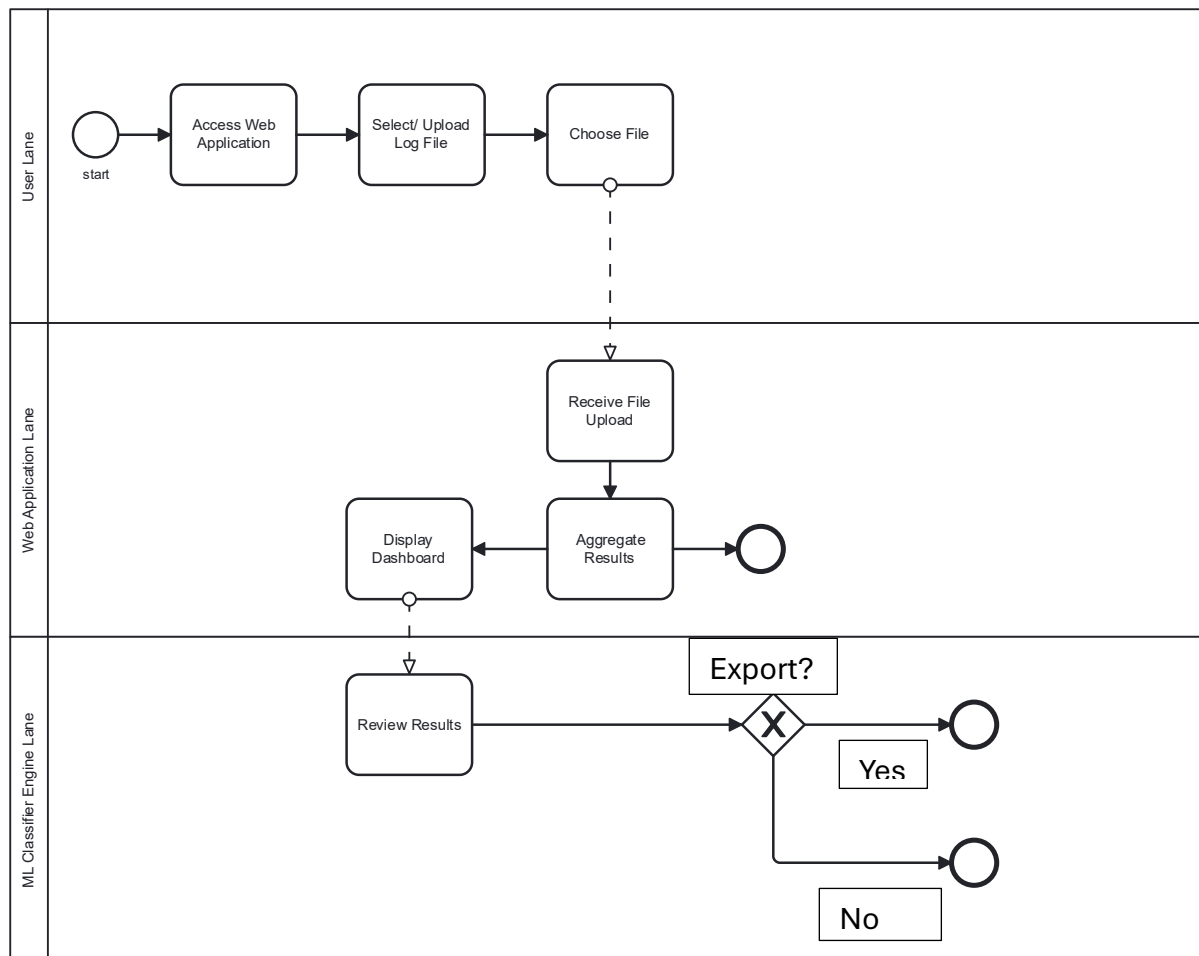


Figure 18(BPMN Model)

5.6 Conclusion

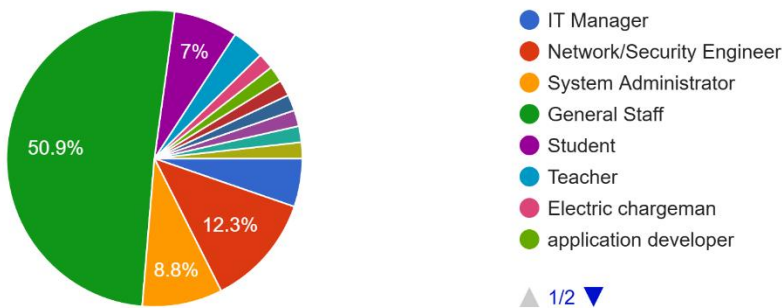
This chapter described the analysis step of AI-based Threat Level Classification System. It incorporated the use case diagram, system flow chart, and BPMN as a tool showing the work flow and relationship among the users and the system components. These models gave a vivid comprehension of the system structure, flow of information and operational edges.

The use case diagram helped in defining major functions like uploading logs, threat classification as well as generation of reports. The flowchart was already a step by step visual depiction of the process of data flowing through the system, whereas the BPMN diagram actualized the process, splitting up the responsibilities between three different lanes, User, Web Application, and ML Classifier Engine.

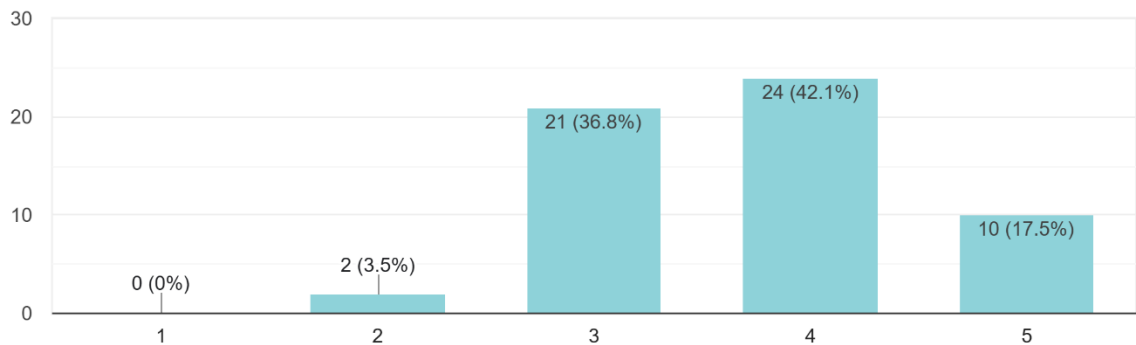
In this analysis, the basic logic behind the system has been explained and acts as a good reference point to design and implement the system in later stages.

5.7 Appendix A – Requirements Specification Document

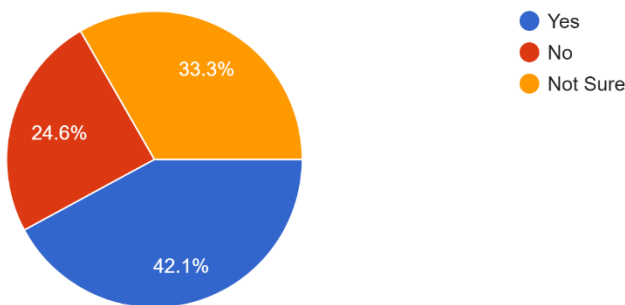
What is your current role in the organization?
57 responses



How familiar are you with cybersecurity tools or threat detection systems?
57 responses

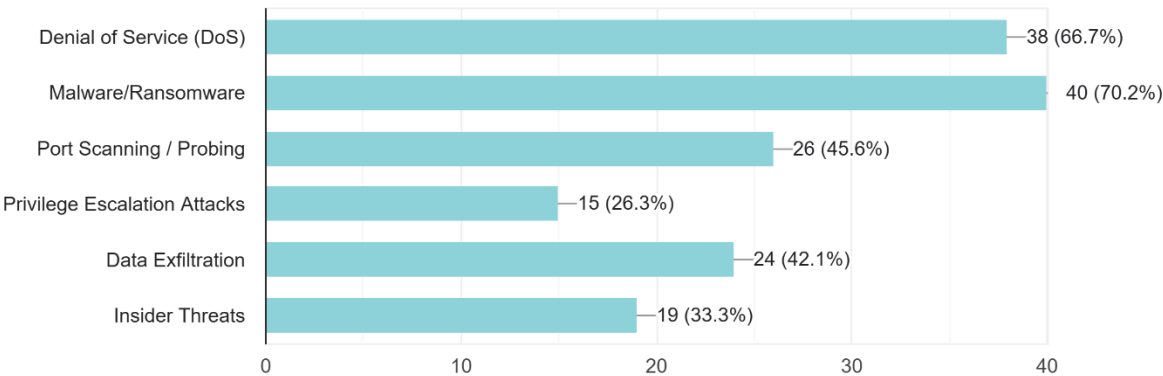


Does your current system provide automated threat classification?
57 responses



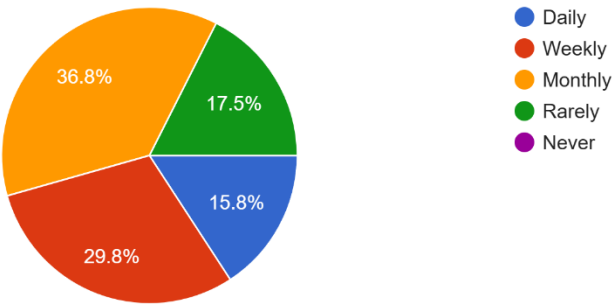
What types of network threats concern your organization the most?

57 responses



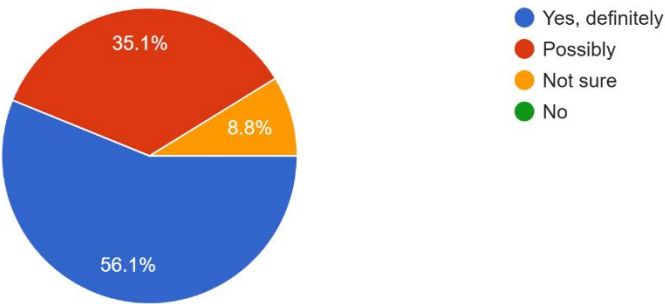
How often do you or your team review network logs or traffic data?

57 responses



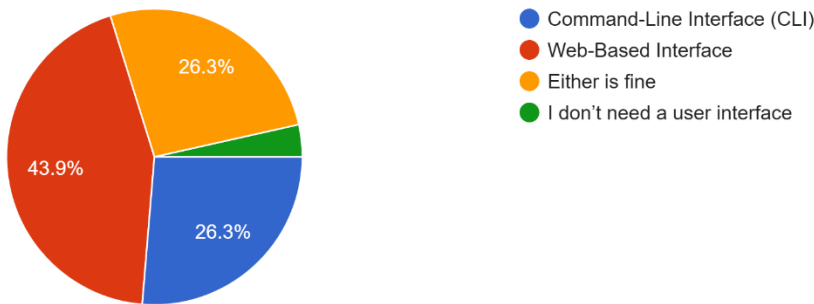
Would an AI-based threat classification system be useful in your environment?

57 responses



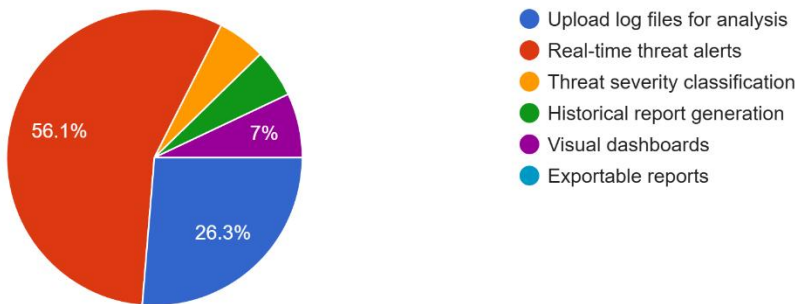
Which type of interface would you prefer for interacting with the system?

57 responses



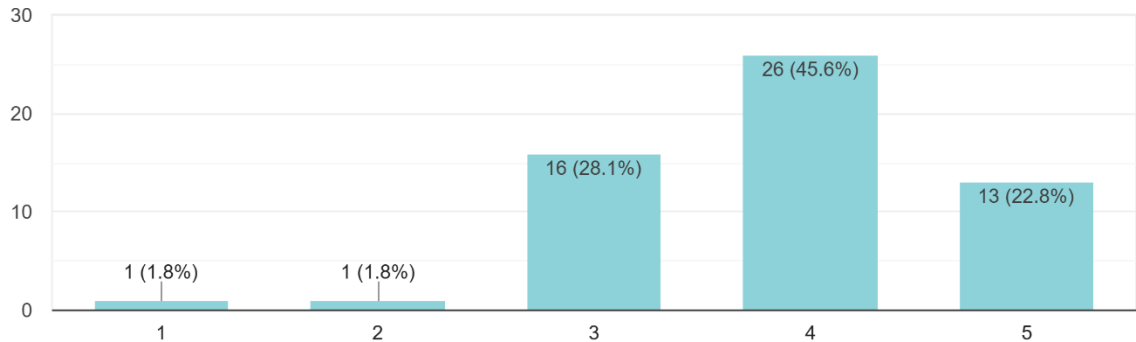
What features would you expect from a network threat classifier?

57 responses

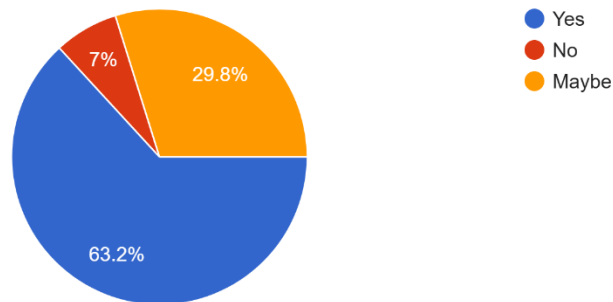


How important is explainability in AI decisions (i.e., knowing why a threat is classified as high/low)?

57 responses



Would you be willing to participate in testing or providing feedback for a prototype?
57 responses



5.8 Appendix B – User Manual

1. System Purpose

This system allows auto-classification of uploaded network log files on Low, Medium or High sets of threats using AI. It eases off manual work and makes IT staff concentrate on important threats.

2. Usage of the System.

A. Step 1: gaining access to the System

Use your web browser.

Copy and paste the system URL, or start the local app, in case it is installed.

Step 2: Embark Your Log File

Click on Upload Log file.

Navigate and find your network log file (it should be of CSV format).

Click “Submit”.

Step 3: File is Processed in the System

The system:

Scans the log information.

Classifies each entry using the AI model that has been trained.

Categorizes them as per the level of threat.

Step 4: See the Results

The next thing after processing, a Dashboard is seen.

It shows:

In total, log entries.

The number of Low, Medium and High threats.

Summary charts (when they exist).

Step 5: Results Export

In case you are interested in saving the results:

Hit the button Export Report.

Download the file in CSV or PDF file.

Step 6: Log Out (Where necessary)

Press logout when your system is log in.

Close the browser.

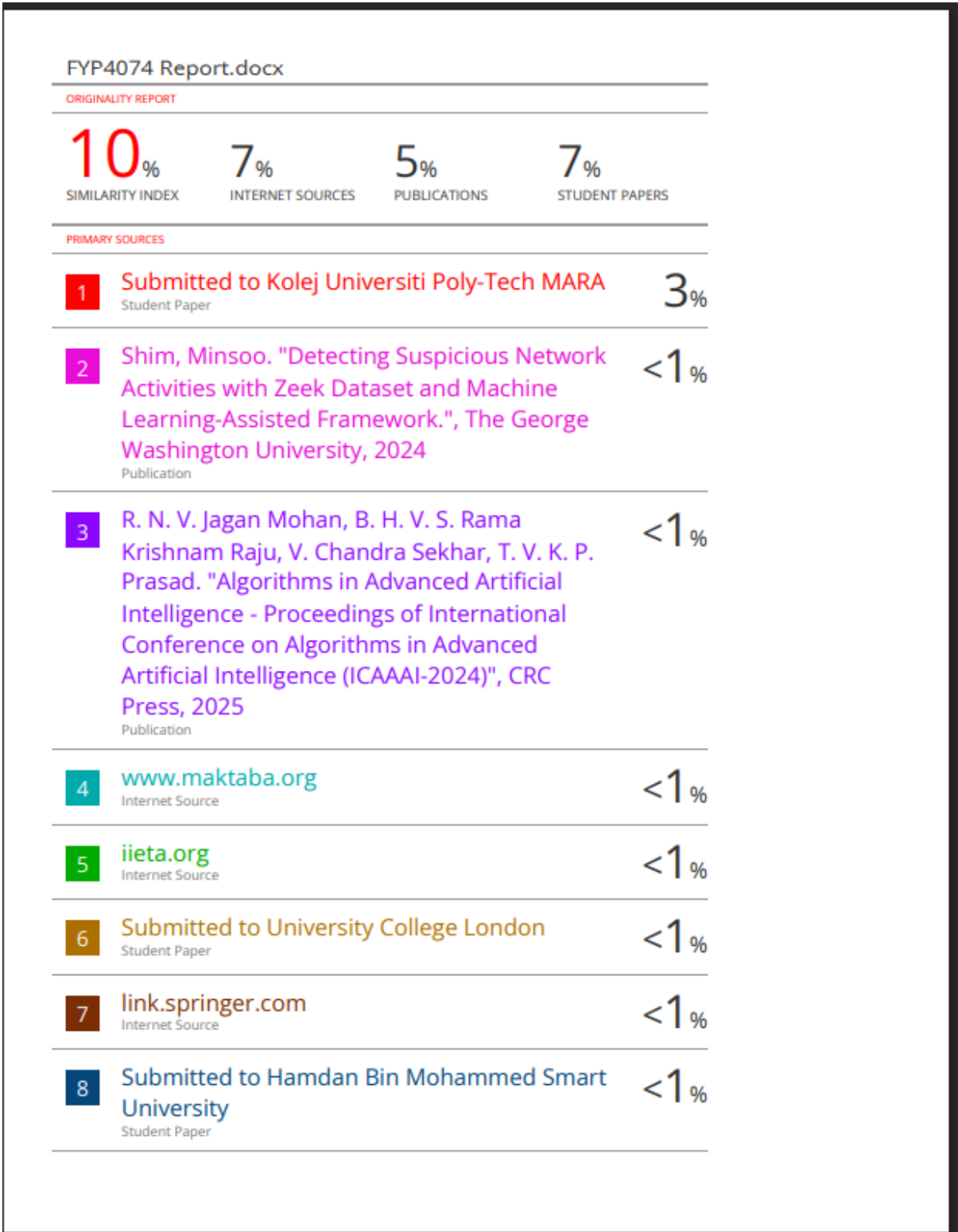
3. Notes and Tips

Never forget to ensure that your CSV file should have the right configuration.

Files that are of a large size can take some time to process.

Apply the results to assist in prioritizing the threats to research first.

5.9 Appendix D– Turnitin Result



9	Submitted to American Intercontinental University Online Student Paper	<1 %
10	ouci.dntb.gov.ua Internet Source	<1 %
11	lup.lub.lu.se Internet Source	<1 %
12	researchcommons.waikato.ac.nz Internet Source	<1 %
13	Submitted to Colorado Technical University Online Student Paper	<1 %
14	Submitted to Delhi Metropolitan Education Student Paper	<1 %
15	irjet.net Internet Source	<1 %
16	Submitted to Chester College of Higher Education Student Paper	<1 %
17	Submitted to Kaplan International Colleges Student Paper	<1 %
18	Submitted to Southern New Hampshire University - Continuing Education Student Paper	<1 %
19	bilakniha.cvut.cz Internet Source	<1 %
20	hdl.handle.net Internet Source	<1 %
21	repository.nwu.ac.za Internet Source	<1 %
22	Submitted to ICTS	

	Student Paper	<1 %
23	www.researchsquare.com Internet Source	<1 %
24	Submitted to North West University Student Paper	<1 %
25	research.ou.nl Internet Source	<1 %
26	www.coursehero.com Internet Source	<1 %
27	www.theseus.fi Internet Source	<1 %
28	zenodo.org Internet Source	<1 %
29	Submitted to Middle East College Student Paper	<1 %
30	blog.enterprisedna.co Internet Source	<1 %
31	openrepository.aut.ac.nz Internet Source	<1 %
32	uir.unisa.ac.za Internet Source	<1 %
33	www.crowdstrike.com Internet Source	<1 %
34	Submitted to Institute of Research & Postgraduate Studies, Universiti Kuala Lumpur Student Paper	<1 %
35	Kaleli, Beliz. "Measuring and Improving the Security of the Web", Boston University, 2025 Publication	<1 %

36	Zheng Zhang, Amit Das, Guan Huang, Sanjeev Baskiyar. "CAT: A simple heterogeneous ensemble learning framework for network intrusion detection", Peer-to-Peer Networking and Applications, 2025 Publication	<1 %
37	fount.aucegypt.edu Internet Source	<1 %
38	lrd.yahooapis.com Internet Source	<1 %
39	openscholar.dut.ac.za Internet Source	<1 %
40	www.corserva.com Internet Source	<1 %
41	Harris, Adam Boyd. "Exploring The Agile System Development Best Practices Cybersecurity Leaders Need to Establish A Cyber-Resilient System: A Phenomenological Study", Colorado Technical University, 2020 Publication	<1 %
42	Marques, André Gonçalves. "Multy-Dimensional Composition by Objectives in Aspect-Oriented Requirements Analysis", Universidade NOVA de Lisboa (Portugal), 2024 Publication	<1 %
43	Trotskyuk, Roman Vladimirovich. "The Enterprise dna : Static and Dynamic Digital Representation of Organizations", Universidade NOVA de Lisboa (Portugal), 2024 Publication	<1 %
44	dokumen.pub Internet Source	<1 %
	etd.lib.metu.edu.tr	

45	Internet Source	<1 %
46	open.library.ubc.ca Internet Source	<1 %
47	utpedia.utp.edu.my Internet Source	<1 %
48	www.episup.com Internet Source	<1 %
49	www.jbdsr.com Internet Source	<1 %
50	"Data Science and Applications", Springer Science and Business Media LLC, 2025 Publication	<1 %
Exclude quotesOff Exclude matchesOff Exclude bibliographyOff		



Page 2 of 62 - AI Writing Overview

Submission ID: trnoid--3618:104003102

0% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

 0 AI-generated only 0%

Likely AI-generated text from a large-language model.

 0 AI-generated text that was AI-paraphrased 0%

Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (it may misidentify writing that is likely AI generated as AI generated and AI paraphrased or likely AI generated and AI paraphrased writing as only AI generated) so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



5.10

Appendix D– Log Book

FYP4074

56

CT206/ BACHELOR OF INFORMATION TECHNOLOGY (HONS) IN CYBER SECURITY



FACULTY OF COMPUTING & MULTIMEDIA (FCOM)

FINAL YEAR PROJECT O1

FYP4074

LOG BOOK

STUDENT'S NAME :MUHAMMAD ADAM MAHFIZ BIN ROSHAYULIZAM

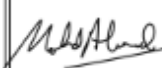
ID NO. :AM2311015306_____

SUPERVISOR :EN. MOHD AKMAL BIN MOHD AZMER_____

PROJECT TITLE : AI-Based Threat Level Classification for Network Logs

CT206/ BACHELOR OF INFORMATION TECHNOLOGY (HONS) IN CYBER SECURITY

STUDENT'S NAME: Muhammad Adam Mahfiz
ID NO.: AM2311015306
SUPERVISOR: Encik Akmal Azmer
PROJECT TITLE: AI-Based Threat Level Classification for Network Logs

Week	Agenda / Activities	Next Agenda / Plan	Supervisor's Signature
Week 1 (18–24 May)	Project briefing, topic confirmation, initial brainstorming on objectives.	Continue background research; draft early problem statements.	
Week 2 (25–31 May)	Gathered more research on threat classification; prepared draft proposal outline. 28 May: Met supervisor for initial consultation and received feedback on project scope and feasibility.	Refine proposal content; start drafting Chapter 1 and problem statement.	
Week 3 (1–7 Jun)	Improved problem statements, drafted Chapter 1 (Introduction) and outlined scope & objectives.	Continue with Chapter 2: Literature Review; collect references.	
Week 4 (8–14 Jun)	Completed Literature Review draft; compared existing related works and summarized methodologies.	Start Methodology draft; outline Agile phases and system flow.	
Week 5 (15–21 Jun)	Developed initial diagrams: Use Case Diagram, Flowchart, and BPMN for analysis chapter.	Prepare slides draft for proposal presentation; revise diagrams based on feedback.	
Week 6 (22–28 Jun)	Finalized proposal document and slides; self-review of chapters 1–3; refined report formatting.	Practice presentation; get feedback from peers if possible.	
Week 7 (29 Jun–5 Jul)	Presented proposal; submitted final proposal report; noted supervisor's final suggestions.	Begin preparing for next FYP phase; review feedback for improvements.	

6. Chapter 6: References

Agile Alliance. (2021). *What is agile software development?* Retrieved from <https://www.agilealliance.org/agile101/>

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for agile software development*. <https://agilemanifesto.org>

Breiman, L. (2001). Random forests. *Machine Learning*, 45*(1), 5–32. <https://doi.org/10.1023/A:1010933404324>

Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials, 18*(2), 1153–1176. <https://doi.org/10.1109/COMST.2015.2494502>

Canonical. (2019). *Visual Studio Code*. Retrieved from <https://snapcraft.io/code>

Fowler, M. (2004). *UML distilled: A brief guide to the standard object modeling language* (3rd ed.). Addison-Wesley Professional.

Grinberg, M. (2018). *Flask web development: Developing web applications with Python* (2nd ed.). O'Reilly Media.

Kent, K., & Grance, T. (2006). *Guide to computer security log management* (NIST Special Publication 800-92). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-92>

Medium. (2020). *Agile figure*. [Image]. Retrieved from [source if available, otherwise general citation]

Microsoft. (2024). *Visual Studio Code* (Version 1.90) [Computer software]. <https://code.visualstudio.com>

Neon Inc. (2024). *Neon Serverless Postgres*. Retrieved May 22, 2024, from <https://neon.tech>

Object Management Group. (2013). *Business Process Model and Notation (BPMN) Version 2.0.2*. Retrieved from <https://www.omg.org/spec/BPMN/2.0.2/>

Open Information Security Foundation. (n.d.). *Suricata*. Retrieved from <https://suricata.io/>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research, 12*, 2825–2830.

Python Software Foundation. (2023). *Python language reference, version 3.11*. Retrieved from <https://docs.python.org/3/reference/index.html>

Reason, J. (2016). *Human error*. Cambridge University Press.

Revathi, S., & Malathi, A. (2013). A detailed analysis on NSL-KDD dataset using various machine learning techniques for intrusion detection. *International Journal of Engineering Research & Technology (IJERT), 2*(12), 1848–1853.

Roesch, M. (1999). Snort - Lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA '99)* (pp. 229–238). USENIX Association.

Sandeep, G. (2025). *Pandas Figure*. [Image]. [Source if available]

Sandeep, G. (2025). *Flask Figure*. [Image]. [Source if available]

Sarker, I. H. (2021). Deep learning for cybersecurity: a comprehensive survey. *ACM Computing Surveys*, 54*(3), 1–36. <https://doi.org/10.1145/3445663>

Sculley, D., O'Keefe, R., & Bar-Yam, O. (2011). Detecting adversarial advertisements in the wild. *In Proceedings of the 17th ACM SIGKDD international conference on Knowledge discovery and data mining** (pp. 568-576). <https://doi.org/10.1145/2020408.2020496>

Splunk Inc. (n.d.). **Splunk enterprise security**. Retrieved from https://www.splunk.com/en_us/products/enterprise-security.html

Stallings, W. (2019). **Cryptography and network security: Principles and practice** (8th ed.). Pearson.

Tavallaee, M., Bagheri, E., Lu, W., & Ghorbani, A. A. (2009). A detailed analysis of the KDD CUP 99 data set. *In *2009 IEEE Symposium on Computational Intelligence for Security and Defense Applications** (pp. 1-6). <https://doi.org/10.1109/CISDA.2009.5356528>

The pandas development team. (2024). **pandas-dev/pandas: Pandas** (Version 2.2.2) [Computer software]. Zenodo. <https://doi.org/10.5281/zenodo.10849611>

Yin, C., Zhu, Y., Fei, J., & He, X. (2017). A deep learning approach for intrusion detection using recurrent neural networks. **IEEE Access*, 5*, 21954–21961. <https://doi.org/10.1109/ACCESS.2017.2762018>